

자구&알고 스터디

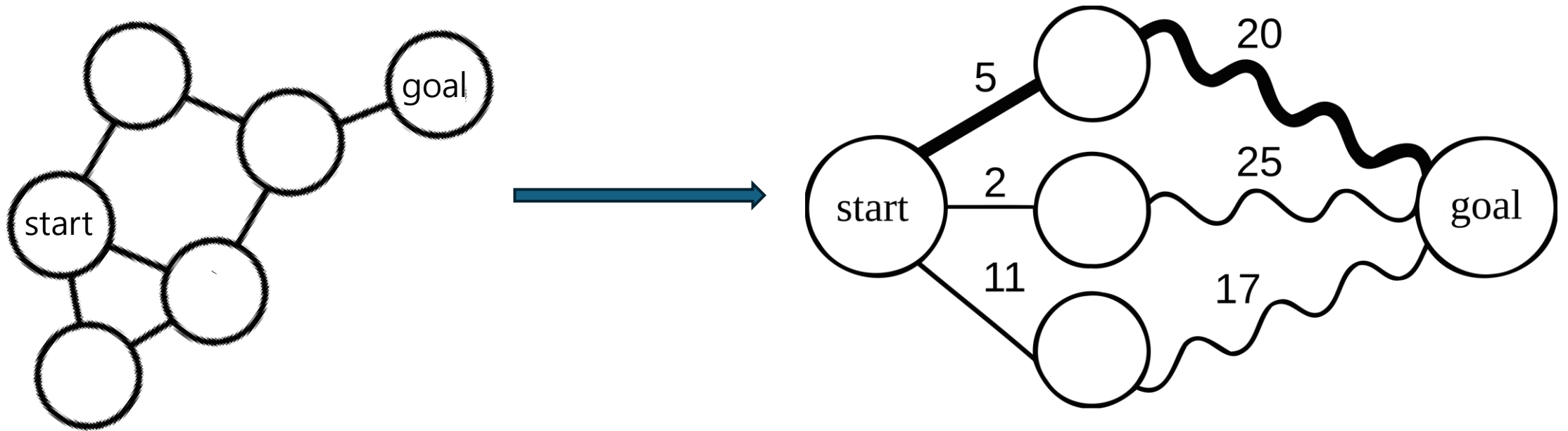
동적 계획법 (Dynamic Programming)

동적 계획법

- 주어진 문제를 더 간단한 문제로 분해한 뒤, 간단한 하위 문제를 해결하고 원래 문제를 해결하는 방식.
- 이를 재귀적으로 접근하면 된다.
- 구체적인 알고리즘이라기보단 문제 해결 접근할 수 있는 유형임.

동적 계획법

- 예시: 최단 거리



동적 계획법

- 최적 부분 구조 (Optimal Substructure)
 - 큰 문제의 최적해가 작은 문제의 최적해를 포함한다.
 - 간단히 말하면 점화식을 찾아야 함
- 예시: 피보나치:
 - $F(N) = F(N - 1) + F(N - 2)$
- 예시: 최단 거리
 - $P(n, k) = \min(P(n, s) + P(s, k))$

동적 계획법

- 중복되는 부분 문제 (Overlapping Subproblem)
 - 같은 작은 문제가 여러 번 반복되어 등장한다.
 - 간단히 말하면 이미 계산한 작은 문제의 결과를 재사용해야 함.
- 이미 계산한 작은 문제의 결과를 재사용하는 방법: "메모이제이션"
 - 이미 계산한 값들을 배열 같은 데에다가 저장하세요.
- 예시: 피보나치
 - $F(N)$ 을 구할 때 $F(x)$ ($x \leq N-2$), 를 반복 계산함
 - 따라서 이전에 계산한 값을 저장해두면 효율적임

동적 계획법

- 유명한 유형들이 많고, 이를 알아두면 편함
 - 백준 밀다 보면 비슷한 문제들 많이 볼 수 있음
- 솔직히 머리 싸움임;;

예시 1

- 피보나치 : <https://www.acmicpc.net/problem/1904>
 - 예시 답변 : <http://boj.kr/18432470cafd4ad089d36e986c38cf6b>

예시 1 해설

- 점화식 : $\text{count}(n) = \text{count}(n - 1) + \text{count}(n - 2)$
 - 이때, $\text{count}(1) = 1, \text{count}(2) = 2$
- 한 배열 memo에 i일 때의 결과를 memo[i]저장하면서 3부터 n까지 계산

예시 2

- 포도주 시식: <https://www.acmicpc.net/problem/2156>
 - 예시 답변 : <http://boj.kr/4bd578782be94743ab1c4f05198438ea>

예시 2 해설

- 점화식 : $dp[i] = \max(dp[i-2], dp[i-3]+a[i-1]) + a[i]$
- base: $dp[0]=0, dp[1]=a[1], dp[2]=a[1]+a[2]$

예시 3

- 평범한 배낭 : <https://www.acmicpc.net/problem/12865>
 - 예시 답안 : <http://boj.kr/2ee43aa071bd44c598cd72f08c1c6f82>

예시 3 해설

- $dp[i][j]$: i 번째 물건까지 고려할 때, j 무게까지의 최대 가치
- 점화식: $dp[i][j] = \max(dp[i-1][j-w]+v, dp[i-1][j], dp[i][j-1])$
- base: $dp[i][j]=0$

예시 3 해설 – Improved Method

- $dp[i]$: i 무게까지의 최대 가치
- 점화식: $dp[i] = \max(dp[i-w]+v, dp[i])$ (뒤에서부터)
- base: $dp[i]=0$